SOFTWARE REQUIREMENTS SPECIFICATION

REAL-TIME CHAT APPLICATION

Document Version: 1.0
Backend: Java 21, Spring Boot 4.1.1
Communication: WebSocket + STOMP
Security: Spring Security + JWT
Persistence: Spring Data JPA, Hibernate, PostgreSQL
Development database: Local PostgreSQL via pgAdmin
Cloud migration target: Neon PostgreSQL

Project Documentation

Document Overview

This document specifies the functional and non-functional requirements, interfaces, database requirements, constraints and future enhancements for a real-time chat application supporting private and group conversations.

Item	Description
Project	Real-Time Chat Application
Version	1.0
Primary users	Registered users and group administrators
Database	PostgreSQL (local development; Neon planned for cloud)
Frontend	Thymeleaf / HTML / CSS / JavaScript (based on current project dependencies)

Table of Contents

• 1. Introduction

• 2. Overall Description

• 3. Functional Requirements

• 4. Non-Functional Requirements

• 5. External Interface Requirements

• 6. Database Requirements

• 7. System Constraints

• 8. Assumptions and Dependencies

• 9. Future Enhancements

• 10. Conclusion

1. Introduction

1.1 Purpose

The purpose of this project is to develop a real-time chat application that enables registered users to communicate through private one-to-one and group conversations. The application provides authentication, username-based user discovery, connection requests, real-time messaging and persistent chat history.

1.2 Scope

• User registration and login using secure authentication.

• Search and discover registered users by username.

• Send, accept, reject and cancel connection requests.

• Establish private connections after a request is accepted.

• Exchange messages in real time using WebSocket and STOMP.

• Persist messages and retrieve conversation history from PostgreSQL.

• Create groups, manage membership and support group messaging.

• Restrict access to conversations to authorized members.

1.3 Intended Audience

This document is intended for developers, testers, project evaluators, academic project guides and system administrators.

1.4 Definitions and Abbreviations

Term	Description
SRS	Software Requirements Specification

Term	Description
JWT	JSON Web Token
REST API	Representational State Transfer API
WebSocket	Full-duplex communication protocol
STOMP	Simple Text Oriented Messaging Protocol
ORM	Object Relational Mapping
JPA	Java Persistence API
DBMS	Database Management System
UI	User Interface

2. Overall Description

2.1 Product Perspective

The application follows a client-server architecture. The frontend communicates with the Spring Boot backend through REST APIs for registration, login, user search and connection management. WebSocket with STOMP supports real-time message delivery. PostgreSQL stores user details, connection requests, connections, conversations, memberships and messages.

2.2 Product Functions

• User registration, authentication and profile management.

• Username-based user search.

• Connection request and acceptance workflow.

• Private conversations and real-time messaging.

• Persistent message storage and retrieval.

• Group creation, administration and group messaging.

• Real-time notifications for requests and messages.

2.3 User Classes

Registered User: Can authenticate, search for users, manage connection requests, chat with accepted connections, create groups and participate in conversations.

Group Administrator: Has additional permissions to manage group information, add or remove members and administer group membership.

3. Functional Requirements

FR-01: User Registration

• The system shall allow new users to register with a username, email address and password.

• The system shall validate required registration information.

• Usernames and email addresses shall be unique.

• Passwords shall be stored as secure BCrypt hashes.

FR-02: User Authentication

• The system shall validate registered users' login credentials.

• The system shall issue a JWT after successful authentication.

• Protected resources shall require authentication.

• Users shall be able to log out; token invalidation or expiry policy shall be defined by implementation.

FR-03: User Search

• Authenticated users shall be able to search for users by username.

• Search shall support partial username matching.

• The current user shall not appear in their own search results.

• Only safe, basic profile information shall be returned.

FR-04: Connection Request Management

• Users shall be able to send connection requests to other users.

• Receivers shall be able to accept or reject incoming requests.

• Users shall be able to view sent and received requests.

• Users shall be able to cancel pending requests.

• The system shall prevent self-requests and duplicate pending requests.

• An accepted request shall establish a connection between the users.

FR-05: Private Messaging

• Connected users shall be able to use private conversations.

• Messages shall be delivered in real time using WebSocket and STOMP.

• Messages shall include sender, conversation, content and timestamp information.

• Messages shall be persisted in PostgreSQL.

• Only conversation members shall be able to access or send messages to that conversation.

FR-06: Chat History

• The system shall retain conversation history in the database.

• Users shall be able to retrieve previous messages after closing the chat or logging in again.

• Message history shall support pagination.

• Messages shall be presented in chronological order.

FR-07: Group Management

• Users shall be able to create group conversations.

• The group creator shall initially be an administrator.

• Authorized administrators shall be able to add or remove members.

• Members shall be able to view group details and leave a group.

• Membership and role information shall be persisted.

FR-08: Group Messaging

• Authorized group members shall be able to exchange real-time messages.

• Group messages shall be stored in PostgreSQL.

• Members shall be able to retrieve group history.

• Non-members shall not be allowed to access group messages.

FR-09: Real-Time Notifications

• The system shall notify users about incoming connection requests.

• The system shall notify users when requests are accepted.

• The system shall deliver messages to active authorized conversation subscribers.

4. Non-Functional Requirements

4.1 Performance

• Messages should be delivered with minimal latency under normal network conditions.

• User search should return results efficiently.

• Chat history shall use pagination to limit unnecessary data retrieval.

• The system should support multiple concurrent users, subject to deployment capacity.

4.2 Security

• Passwords must be stored using secure password hashing.

• JWT shall be used for authenticated API access.

• REST endpoints and WebSocket connections/subscriptions shall enforce authentication and authorization.

• Users shall not access conversations to which they do not belong.

• Only authorized group administrators shall manage group membership.

• Database credentials shall not be committed to source control.

4.3 Reliability

• Messages shall be persisted before being broadcast to recipients.

• Stored messages shall remain available after application restarts, subject to database retention and backup policies.

• The system shall handle unexpected WebSocket disconnections gracefully.

• Database errors shall be handled without exposing credentials or sensitive details.

4.4 Scalability

• Database queries should use appropriate indexes.

• Message retrieval shall be paginated.

• The design should allow migration from local PostgreSQL to a managed cloud PostgreSQL service such as Neon.

• A production deployment with multiple backend instances may require a shared message broker rather than the in-memory simple broker.

4.5 Usability

• The interface should be clear and easy to navigate.

• Users should be able to move between connections, private chats and groups.

• The system should provide understandable validation and error messages.

5. External Interface Requirements

5.1 User Interface

• Registration page

• Login page

• User dashboard

• User search page

• Connection requests page

• Connections list

• Private chat window

• Group creation and management page

• Group chat window

• User profile page

5.2 Software Interfaces

Component	Technology
Backend	Java 21, Spring Boot 4.1.1
Frontend (current dependency)	Thymeleaf, HTML, CSS, JavaScript
Database	PostgreSQL
ORM	Spring Data JPA, Hibernate
Authentication	Spring Security, JWT
Real-time communication	WebSocket, STOMP
Build tool	Maven
Database administration	pgAdmin
API testing	Postman

5.3 Communication Interfaces

• HTTP/HTTPS shall be used for REST API communication.

• WebSocket shall be used for real-time, bidirectional communication.

• STOMP shall provide message routing over WebSocket.

• JDBC shall be used for PostgreSQL connectivity.

6. Database Requirements

The proposed relational database contains six core entities. A shared Conversation and Message model is used for both private and group chats.

Entity	Purpose
USER	Stores registered user identity, login and profile information.
CONNECTION_REQUEST	Stores sender, receiver, request status and creation time.
USER_CONNECTION	Stores accepted user-to-user connections.
CONVERSATION	Represents a PRIVATE or GROUP conversation.
CONVERSATION_MEMBER	Associates users with conversations and stores membership roles.
MESSAGE	Stores message content, sender, conversation and timestamp.

6.1 Recommended Constraints

• USER.username and USER.email shall be unique.

• A user cannot send a connection request to themselves.

• Duplicate pending requests shall be prevented.

• USER_CONNECTION shall prevent duplicate connections; store each pair once using canonical user ID ordering.

• CONVERSATION_MEMBER shall have a unique constraint on (conversation_id, user_id).

• CONVERSATION.type shall be restricted to PRIVATE or GROUP.

• A message sender must be a member of the associated conversation.

• A private conversation should be unique for a pair of users; enforce this with application logic and an appropriate database strategy.

7. System Constraints

• Users must register and authenticate before using protected messaging features.

• Only accepted connections may start private conversations.

• Only conversation members may access conversation messages.

• Group membership changes shall follow group authorization rules.

• An active network connection is required for live communication.

• PostgreSQL must be available for persistent storage.

8. Assumptions and Dependencies

• Users have access to a supported web browser.

• The Spring Boot backend is reachable over the network.

• PostgreSQL is installed and configured for local development.

• WebSocket communication is supported by the client environment.

• Users have valid registered accounts.

• The initial release supports text messages; media sharing may be added later.

9. Future Enhancements

• Image, video and document sharing using object/cloud storage.

• Voice and video calling.

• Message editing and deletion.

• Read receipts and typing indicators.

• Online/offline presence.

• Push notifications and unread message counts.

• Message search and reactions.

• End-to-end encryption.

10. Conclusion

The Real-Time Chat Application aims to provide a secure and reliable communication platform supporting private and group messaging. By combining Spring Boot, Spring Security, JWT, WebSocket, STOMP and PostgreSQL, the system provides real-time communication with persistent message storage. Its modular architecture allows additional features to be introduced in future versions.

End of Document